SE 3242: Android Application Development

Week 2: Kotlin Essentials
Session 1: Functions, Collections & More

Engr. Daniel MOUNE
moune.daniel@ictuniversity.edu.cm
ICT University, Cameroon Campus, Yaoundé
P.O. Box 526, 1 Avenue Dispensaire Messassi, Zoatoupsi

Imparting ICTs in all academic disciplines

February 24, 2026

Quick Recap – What We Learned last week

Variables & Types

- `val` (read-only) vs `var` (mutable)
- Type inference: `val name = "Alice"`
- Basic types: `Int`, `Double`, `Boolean`, `String`
- Explicit conversions: `.toInt()`, `.toString()`

Null Safety

- Nullable types: `String?`
- Safe call: `?.`
- Elvis operator: `?:`
- Not-null assertion: `!!` (use with caution!)

Conditionals

`if` is an expression: `val max = if (a > b) a else b`

`when` is a powerful switch: `when (x) { 1 -> "one" else -> "many" }`

"Let's build on this foundation with functions!"

Today's Roadmap – Kotlin Functions & Collections

Part 1: Functions – The Basics (30 min)

- Function syntax
- Parameters and return types
- Expression body vs block body

Part 2: Advanced Functions (30 min)

- Default and named arguments
- Varargs, infix functions
- Extension functions

Part 3: Collections (45 min)

- Lists, Sets, Maps
- Iteration and basic operations

Part 4: Lambdas & Higher-Order Functions (45 min)

- Function types
- Lambdas, `it`
- Common higher-order functions (`map`, `filter`)

Part 5: Project Milestone 2 (10 min)

"Recap from last week: we'll build on variables, null safety, and conditionals."

Functions – Reusable Blocks of Code

Function Syntax

```
fun functionName(param1: Type1, param2: Type2): ReturnType {
    // body
    return result
}
```

Simple function

```
fun greet(name: String): String {
    return "Hello, $name"
}
// Usage: greet("Alice") -> "Hello, Alice"
```

No parameters, no return

```
fun sayHello() {
    println("Hi!")
}
// Return type is Unit (like void)
```

Expression Body – Concise Functions

Single-expression functions

If a function returns a single expression, you can omit the braces and use =.

```
fun add(a: Int, b: Int): Int = a + b
```

Return type can be inferred:

```
fun add(a: Int, b: Int) = a + b    // Inferred return type Int

fun add(a: Float, b: Float) = a + b    // Inferred return type Float
```

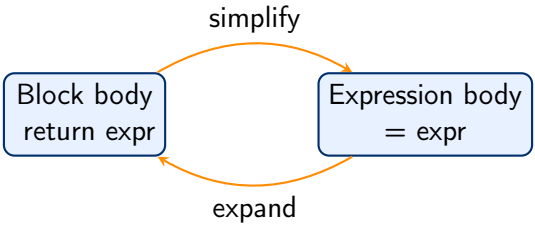
Block body (explicit return)

```
fun max(a: Int, b: Int): Int {
    return if (a > b) a else b
}
```

Expression body (concise)

```
fun max(a: Int, b: Int) = if (a > b) a else b
```

Expression Body – Concise Functions



Flexible Functions – Default & Named Arguments

Default arguments

You can provide default values for parameters, making them optional when calling.

```
fun greet(name: String = "Guest", greeting: String = "Hello") {
    println("$greeting, $name")
}
```

Calling with defaults

```
greet()
// Hello, Guest
greet("Alice")
// Hello, Alice
greet("Bob", "Hi")          // Hi, Bob
```

Named arguments

Specify argument names for clarity or to skip some defaults.

```
greet(greeting = "Good morning", name = "Cha
// Good morning, Charlie
```

"Named arguments are great when functions have many parameters!"

Varargs – Passing Any Number of Arguments

Vararg Parameter

Use the `vararg` modifier to accept a variable number of arguments.

```
fun printAll(vararg messages: String) {
    for (msg in messages) println(msg)
}
```

Calling with multiple arguments

```
printAll("Hello", "Kotlin", "World")
// Prints each string on a new line
```

Spread operator *

Pass an array as individual arguments.

```
val words = arrayOf("A", "B", "C")
printAll(*words)
```

"Varargs make functions flexible – like `printf` in C."

Infix Functions – Calling Without Dot and Parentheses

Infix Notation

Functions marked with `infix` can be called without the dot and parentheses, if they have exactly one parameter.

```
infix fun Int.times(str: String) = str.repeat(this)
```

Normal call

```
val result = 3.times("Hi")    // HiHiHi
```

Infix call

```
val result = 3 times "Hi"     // HiHiHi
```

Common infix functions in Kotlin

`to` creates a pair: `1 to "one"` → `Pair(1, "one")`

"Infix functions make domain-specific languages (DSLs) elegant!"

Extension Functions – Adding Methods to Existing Classes

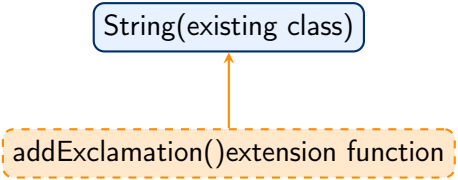
What are extension functions?

You can add new functions to a class without inheriting from it.

```
fun String.addExclamation() = this + "!"
```

Usage

```
val greeting = "Hello".addExclamation()
println(greeting)    // Hello!
```



With generics

```
fun <T> List<T>.secondOrNull(): T? =
    if (size >= 2) this[1] else null
```

Collections – Storing Multiple Values

Immutable vs Mutable

Kotlin distinguishes between read-only and mutable collections.

List

Ordered, allows duplicates.

```
val list = listOf(1, 2, 3)
// immutable
val mList = mutableListOf(1, 2)
// mutable
mList.add(3)
```

Set

Unordered, unique elements.

```
val set = setOf(1, 2, 2, 3)
// {1,2,3}
val mSet = mutableSetOf(1, 2)
mSet.add(3)
```

Map

Key-value pairs.

```
val map = mapOf(1 to "one", 2 to "two"
val mMap = mutableMapOf(1 to "one")
mMap[2] = "two"
```

"Prefer immutable collections by default – use mutable only when needed."

Working with Collections – Common Operations

Iteration

```
val list = listOf("A", "B", "C")
for (item in list) println(item)
list.forEach { println(it) }
```

Filtering

```
val numbers = listOf(1, 2, 3, 4, 5)
val even = numbers.filter { it % 2 == 0 }
// [2, 4]
```

Mapping

```
val squares = numbers.map { it * it }
// [1, 4, 9, 16, 25]
```

Any, all, none

```
numbers.any { it > 3 }   // true
numbers.all { it > 0 }   // true
numbers.none { it < 0 }  // true
```

"Kotlin's standard library provides many functional helpers!"

Function Types – Functions as First-Class Citizens

Declaring a function type

Syntax: `(ParameterTypes) -> ReturnType`

```
val greeting: (String) -> String = { name -> "Hello, $name" }
```

`greeting` is a variable that holds a function.

Calling a function variable

```
println(greeting("Alice"))
// Hello, Alice
```

Type inference

```
val greeting = { name: String -> "Hello, $na
```

Kotlin infers the type.

"Functions can be passed around just like numbers or strings."

Lambda Syntax – Writing Anonymous Functions

Lambda expression

Enclosed in curly braces: `{ parameters -> body }`

```
val sum = { x: Int, y: Int -> x + y }
```

The last expression is the return value.

Single parameter – `it`

For a single parameter, you can omit the name and use `it`.

```
val numbers = listOf(1, 2, 3)
numbers.forEach { println(it) }
```

```
{ x: Int, y: Int -> x + y }
```

parameters body (last line is result)

Trailing lambda

If the last parameter is a function, you can move the lambda outside.

```
numbers.map { it * 2 }
// Equivalent to: numbers.map({ it * 2 })
```

Higher-Order Functions – Functions That Take Functions

Definition

A higher-order function is a function that takes another function as a parameter or returns a function.

Example: custom repeat

```
fun repeat(times: Int, action: (Int) -> Unit) {
    for (i in 0 until times) action(i)
}
```

Usage:

```
repeat(3) { println("Hello #$it") }
```

Function returning a function

```
fun makeMultiplier(factor: Int): (Int) -> Int
    { x -> x * factor }
```

Usage

```
val double = makeMultiplier(2)
println(double(5))  // 10
```

Power of Lambdas: `map`, `filter`, `fold`

`map`

Transform each element.

```
val numbers = listOf(1, 2, 3)
val doubled = numbers.map { it * 2 }
// [2, 4, 6]
```

`filter`

Keep elements matching a predicate.

```
val even = numbers.filter { it % 2 == 0 }
// [2]
```

`fold`

Accumulate a value.

```
val sum = numbers.fold(0) { a
// 6
```

Chaining

```
val result = listOf(1, 2, 3, 4)
    .filter { it > 2 }
    .map { it * 10 }
    .fold("") { acc, i -> acc + i.toString() }
// "3040"
```

Project Milestone 2: Enhance Your App Logic

Objective

Add functions and collection processing to your app's data model.

Tasks

- Create at least two functions that operate on your data class (e.g., validation, formatting).
- Use a higher-order function (e.g., `map`, `filter`, `forEach`) on a list of your objects.
- Write a small demonstration in `main()` that showcases:
  - A lambda passed to a custom higher-order function.
  - A collection operation (e.g., filter a list of items).

Submission

Due: **Next Monday 23:59**

Add to your existing `Milestone1_YourName.kt` file (or create a new file) and submit on LMS.

"This milestone ensures you can apply functional programming concepts in Kotlin."

Kotlin vs Dart – Defining Functions

Kotlin

```kotlin
// Block body
fun greet(name: String): String {
    return "Hello, $name"
}

// Expression body
fun greet(name: String) = "Hello, $name"
```

Dart

```dart
// Block body
String greet(String name) {
  return "Hello, $name";
}

// Arrow syntax
String greet(String name) => "Hello,
    $name";
```

Observations

- Kotlin uses `fun`, Dart uses explicit return type or `Function` keyword.
- Both support concise single-expression syntax: Kotlin with =, Dart with =>.
- Kotlin's expression body infers return type; Dart's arrow syntax requires explicit return type or `var`/ `final`.

Kotlin vs Dart – Lambdas (Anonymous Functions)

Kotlin

```kotlin
val sum = { x: Int, y: Int -> x + y }
// Usage: sum(3, 4)

val numbers = listOf(1, 2, 3)
numbers.forEach { println(it) }

val doubled = numbers.map { it * 2 }
```

Dart

```dart
final sum = (int x, int y) => x + y;
// Usage: sum(3, 4)

final numbers = [1, 2, 3];
numbers.forEach((n) => print(n));

final doubled = numbers.map((n) => n * 2)
    .toList();
```

Key Differences

- Kotlin uses curly braces {} and -> for lambdas; Dart uses parentheses and => for single-expression lambdas (or block bodies with {}).
- Kotlin has `it` for single-parameter lambdas; Dart requires explicit parameter names.
- Both support trailing lambda syntax (Kotlin more seamlessly; Dart requires parentheses if the lambda is the last argument)

Kotlin vs Dart – `map`, `filter`, `fold`

Kotlin

```kotlin
val numbers = listOf(1, 2, 3, 4)
val even = numbers.filter { it % 2 == 0 }
val squared = numbers.map { it * it }
val sum = numbers.fold(0) { acc, i -> acc +
```

Dart

```dart
final numbers = [1, 2, 3, 4];
final even = numbers.where((n) => n % 2
    == 0);
final squared = numbers.map((n) => n * n)
    ;
final sum = numbers.fold(0, (acc, i) =>
    acc + i);
```

Note: Dart uses `where` instead of `filter`, and `fold` parameter order differs.

"Both languages embrace functional collection processing – the concepts are identical, only syntax varies."

Kotlin vs Dart – Named and Default Arguments

Kotlin

```kotlin
fun greet(
    name: String = "Guest",
    greeting: String = "Hello"
) {
    println("$greeting, $name")
}

// Calls
greet()
greet(name = "Alice")
greet(greeting = "Hi", name = "Bob")
```

Dart

```dart
void greet({String name = "Guest", String
    greeting = "Hello"}) {
  print("$greeting, $name");
}

// Calls
greet()
greet(name: "Alice")
greet(greeting: "Hi", name: "Bob")
```

Dart uses curly braces {} for named parameters.

Similarities

- Both support default values.
- Both allow calling with named arguments in any order.

Kotlin vs Dart – Adding New Functionality to Existing Classes

Kotlin (Extension Functions)

```kotlin
fun String.addExclamation() = this + "!"

val text = "Hello"
println(text.addExclamation()) // Hello!
```

Dart (Extension Methods)

```dart
extension StringExtension on String {
  String addExclamation() => this + '!';
}

final text = "Hello";
print(text.addExclamation()); // Hello!
```

Dart requires an `extension` block.

"Both languages let you extend classes without inheritance – a powerful tool for API design."

Summary – Leverage Your Dart Knowledge

Key Takeaways

- Kotlin and Dart share many modern language features:
  - Concise function syntax (expression bodies / arrow syntax).
  - Lambdas and higher-order functions.
  - Named and default arguments.
  - Extension functions / methods.
- The main differences are syntactic:
  - Kotlin uses `fun`, Dart uses return type or `Function`.
  - Kotlin lambdas: `{ it -> ... }`; Dart: `(it) => ...`
  - Kotlin collection functions: `filter`, `map`; Dart: `where`, `map`.
- Understanding these parallels makes learning Kotlin faster and easier.

Next Steps

Apply these concepts in your project milestone – you already know the patterns!

Live Coding: Filtering and Transforming with Lambdas

Problem Statement

Given a list of numbers, perform the following operations using lambda expressions:

1. Filter out numbers less than or equal to 5.
2. Square each remaining number.
3. Print each squared number.

Sample Data

```
val numbers = listOf(1, 4, 7, 3, 9, 2, 8)
```

Step-by-Step Solution

```
numbers
    .filter { it > 5 }          // Keep numbers > 5
    .map { it * it }             // Square each
    .forEach { println(it) }     // Print results
```

Exercise 1: Build Your Own Higher-Order Function

Task

Write a function `processList` that takes a list of integers and a lambda (Int) -> Boolean, and returns a new list containing only the elements that satisfy the predicate.

Function Signature

```
fun processList(
    numbers: List<Int>,
    predicate: (Int) -> Boolean
): List<Int> {
    // your code here
}
```

Test with

```
val nums = listOf(1, 2, 3, 4, 5, 6)
val even = processList(nums) { it % 2 == 0 }
println(even)   // [2, 4, 6]
```

Exercise 2: Transforming Between Collection Types

Task

Given a list of strings, create a map where the keys are the strings and the values are their lengths. Then print only the entries where the length is greater than 4.

Sample Data

```
val words = listOf("apple", "cat", "banana", "dog", "elephant")
```

Expected Output

```
apple has length 5
banana has length 6
elephant has length 7
```

Hint: Use `associateWith` to create the map, then `filter` and `forEach`.

Exercise 3: Complex Data Processing

Task

You have a list of people with names and ages. Find the average age of people whose names start with the letter 'A' or 'B'. Print the result rounded to one decimal place.

Data Class and Sample

```
data class Person(val name: String, val age: Int)

val people = listOf(
    Person("Alice", 25),
    Person("Bob", 30),
    Person("Charlie", 35),
    Person("Anna", 22),
    Person("Ben", 28)
)
```

Exercise 3: Complex Data Processing

Steps

1. Filter people whose name starts with 'A' or 'B'.
2. Extract ages.
3. Calculate average.
4. Format and print.

Session 2 Wrap-Up

What We Covered Today

- **Functions**: syntax, expression body, default/named arguments, varargs
- **Advanced Functions**: infix notation, extension functions
- **Collections**: lists, sets, maps, immutability
- **Lambdas & Higher-Order Functions**: function types, `map`, `filter`, `fold`
- **Kotlin vs Dart**: comparing syntax and concepts

Project Milestone 2 Reminder

Enhance your app logic with functions and collection processing. Due next Monday.

Next Session

Week 2, Session 2: Object-Oriented Programming in Kotlin – classes, inheritance, interfaces, data classes, and sealed classes.

"Any questions before we wrap up?"